

A gamified introduction to Python Programming

Lecture 17

Introduction to Objects and OOP

Matthieu DE MARI – Singapore University of Technology and Design



SINGAPORE UNIVERSITY OF
TECHNOLOGY AND DESIGN



Outline (Chapter 17)

- What are **objects** and **classes**? How do they relate to **dictionaries**?
- What are **attributes** and **methods** in a class?
- What is the **__init__** constructor method?
- What is the **__dict__** special attribute?
- What are **special methods** in Python?
- What is the “**has-a**” relationship in OOP?
- (*What is the “**is-a**” relationship in OOP?*)
- (*What are the different **attributes** **privacies**?*)
- (*What are **setters**, **getters** and **properties**?*)

What is object-oriented programming?

Definition (**Object-Oriented Programming**):

Object-oriented programming (OOP) is a programming language **paradigm** organized around the creation of **custom objects**.

- Sometimes, the **concepts** we need for our programs cannot be described using only the basic int/float/str/list/dict types.
- We could assemble variables that relate to the same concept into dictionaries (what we described as object-oriented thinking), we often prefer to **create our own custom types/objects**.
- These types are created using the **class keyword**, and might come with their own list of **attributes** and **methods**.

Our toy example: an RPG main protagonist

Let us say we would like to code a video game and design our main character/protagonist.

- Our **main character** can be represented as a **custom class object**.
- It would have several **attributes**, e.g.
 - A name (string type, 'Sir Meowsalot')
 - A class (string type, 'Warrior')
 - Some lifepoints (int types, 100)
 - And many other attributes (intelligence, strength, speed, armor, etc.)



Class attributes

Let us say we would like to code a video game and design our main character/protagonist.

- Our **main character** can be represented as a **custom class object**.
- It would have several **attributes**, e.g.
 - A name (string type, 'Sir Meowsalot')
 - A class (string type, 'Warrior')
 - Some lifepoints (int types, 100)
 - And many other attributes (intelligence, strength, speed, armor, etc.)

```
1 class Hero:
2     # Hero's name
3     name = "Sir Meowsalot"
4     # Hero's class
5     hero_class = "Warrior"
6     # Hero's maximal lifepoints
7     maximal_lifepoints = 100
8     # Hero's current lifepoints
9     current_lifepoints = 100
```

Class attributes

To give you an analogy:

- The **def** keyword was used to introduce new functions.
- The **class** keyword is used to introduce a **new type of variable** to Python.

Here we defined the blueprint for a new class “*Hero*”.

Later, we created a variable *my_cat_hero*, which is a variable of type “*Hero*”.

```
1 class Hero:
2     # Hero's name
3     name = "Sir Meowsalot"
4     # Hero's class
5     hero_class = "Warrior"
6     # Hero's maximal lifepoints
7     maximal_lifepoints = 100
8     # Hero's current lifepoints
9     current_lifepoints = 100
```

```
1 my_cat_hero = Hero()
```

```
1 print(my_cat_hero)
```

```
<__main__.Hero object at 0x000001D8D3459CC0>
```

Class attributes

Definition (object **attributes**):

In Python, **object attributes** are variables that are associated with an object and belong to a class.

These attributes store values and data that are relevant to the object.

They are defined within the class block, as variables and can later be accessed using **the . notation** on **our object variable** (like when we did `a.shape` in Numpy).

```
1 class Hero:
2     # Hero's name
3     name = "Sir Meowsalot"
4     # Hero's class
5     hero_class = "Warrior"
6     # Hero's maximal lifepoints
7     maximal_lifepoints = 100
8     # Hero's current lifepoints
9     current_lifepoints = 100
```

```
1 my_cat_hero = Hero()
```

```
1 print(my_cat_hero)
```

```
<__main__.Hero object at 0x000001D8D3459CC0>
```

```
1 print(my_cat_hero.name)
```

```
Sir Meowsalot
```

Class methods

On top of a class **attributes**, we can also define **class methods**.

- **Methods:** functions which apply on our custom object.
- **Methods** may use/modify some of the objects **attributes**.

```
1 class Hero:
2
3     # Hero's name
4     name = "Sir Meowsalot"
5     # Hero's class
6     hero_class = "Warrior"
7
8     def meow(self):
9         '''
10            A first method.
11            '''
12         print("meow.")
13
14
15     def loud_meow(self):
16         '''
17            A second method, calling some attributes of the class.
18            '''
19         print("{} SAYS MEOW.".format(self.name))
```


The **self** keyword

```
1 lst = [0,1,2,3]
2 lst.append(4)
```

In all the methods we used a keyword, called **self**.

- **Self** simply refers to the object on which the method applies.
- In the case of *lst.append(4)*, **self** designated the list *lst* on which *append()* was applied.

The **self** keyword

In all the methods we used a keyword, called **self**.

- **Self** simply refers to the object on which the method applies.
- In the case of *my_cat_hero.meow()*, **self** will refer to our newly typed variable *my_cat_hero*.

```
1  lst = [0,1,2,3]
2  lst.append(4)
```

```
1  my_cat_hero = Hero()
```

```
1  my_cat_hero.meow()
```

meow.

```
1  my_cat_hero.loud_meow()
```

Sir Meowsalot SAYS MEOW.

```
20  def loud_meow(self):
21      '''
22      A second method, calling some attributes of the class.
23      '''
24      print("{} SAYS MEOW.".format(self.name))
```

Class special methods

Definition (**special methods**):

A custom class may also have **special methods**.

- These methods have fixed names and are written with **double underscores (__)** before and after their names.
- These methods do something special when some basic operations (+, *, len(), etc.) are applied to our object.



The most important special method: the `__init__` constructor method

Definition (the `__init__` method):

The most important special method is `__init__`, which is called behind the scenes every time an object is created.

- This is called the **constructor** method of the class.
- When we run the operation `my_cat_hero = Hero()`, Python runs whatever is in the `__init__` method of the *Hero* class.

```
1 class Hero:
2
3     def __init__(self):
4         '''
5         Constructor function for the Hero class.
6         '''
7         # Hero's name
8         self.name = "Sir Meowsalot"
9         # Hero's class
10        self.hero_class = "Warrior"
11        # Hero's maximal lifepoints
12        self.maximal_lifepoints = 100
13        # Hero's current lifepoints
14        self.current_lifepoints = 100
15
16        # Print (to let the user know)
17        print("A new hero has been created!")
```

```
1 my_cat_hero = Hero()
```

A new hero has been created!

The `__init__` constructor vs. « trash » initialization

- It is often preferable to define and use the `__init__` method!
- It is considered **good practice**.
- It also allows for more **actions** on **initialization**.

```
1 class Hero:
2
3     # Hero's name
4     name = "Sir Meowsalot"
5     # Hero's class
6     hero_class = "Warrior"
7     # Hero's maximal lifepoints
8     maximal_lifepoints = 100
9     # Hero's current lifepoints
10    current_lifepoints = 100
```

```
1 class Hero:
2
3     def __init__(self):
4         '''
5         Constructor function for the Hero class.
6         '''
7         # Hero's name
8         self.name = "Sir Meowsalot"
9         # Hero's class
10        self.hero_class = "Warrior"
11        # Hero's maximal lifepoints
12        self.maximal_lifepoints = 100
13        # Hero's current lifepoints
14        self.current_lifepoints = 100
15
16        # Print (to let the user know)
17        print("A new hero has been created!")
```

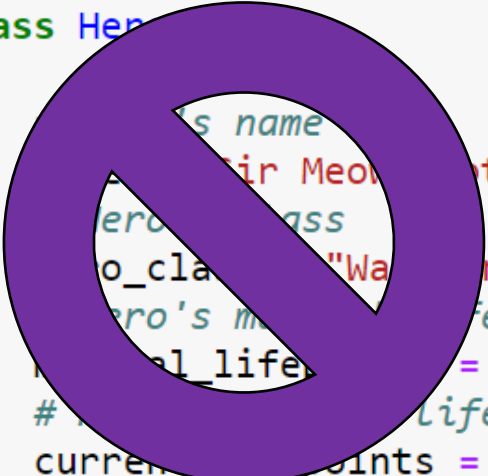
```
1 my_cat_hero = Hero()
```

A new hero has been created!

The `__init__` constructor vs. « trash » initialization

- It is often preferable to define and use the `__init__` method!
- It is considered **good practice**.
- It also allows for more **actions** on **initialization**.

```
1 class Hero:
2     def __init__(self):
3         ...
4         Constructor function for the Hero class.
5         ...
6         # Hero's name
7         self.name = "Sir Meowsalot"
8         # Hero's class
9         self.hero_class = "Warrior"
10        # Hero's maximal lifepoints
11        self.maximal_lifepoints = 100
12        # Hero's current lifepoints
13        self.current_lifepoints = 100
```



```
1 class Hero:
2
3     def __init__(self):
4         ...
5         Constructor function for the Hero class.
6         ...
7         # Hero's name
8         self.name = "Sir Meowsalot"
9         # Hero's class
10        self.hero_class = "Warrior"
11        # Hero's maximal lifepoints
12        self.maximal_lifepoints = 100
13        # Hero's current lifepoints
14        self.current_lifepoints = 100
15
16        # Print (to let the user know)
17        print("A new hero has been created!")
```

```
1 my_cat_hero = Hero()
```

A new hero has been created!

While we're at it, let us talk about default values in methods and functions

- We can also define **inputs** and **default values** in our **methods** and **special methods**.
- Here `__init__` expects a **name** and a **hero_class**, as **mandatory inputs**.

```
1 class Hero:
2
3     def __init__(self, name, hero_class, maximal_lifepoints = 100):
4         """
5         Constructor function for the Hero class.
6         """
7         # Hero's name
8         self.name = name
9         # Hero's class
10        self.hero_class = hero_class
11
12        # Hero's maximal lifepoints
13        # (initialize as maximal_lifepoints)
14        self.maximal_lifepoints = maximal_lifepoints
15
16        # Hero's current lifepoints
17        # (initialize as maximal_lifepoints)
18        self.current_lifepoints = maximal_lifepoints
```

```
1 my_first_hero = Hero(name = "Sir Meowsalot", hero_class = "Warrior")
2 print(my_first_hero.__dict__)
```

```
{'name': 'Sir Meowsalot', 'hero_class': 'Warrior', 'maximal_lifepoints': 100, 'current_lifepoints': 100}
```

```
1 my_second_hero = Hero(name = "Lord Mustache", hero_class = "Mage", maximal_lifepoints = 50)
2 print(my_second_hero.__dict__)
```

```
{'name': 'Lord Mustache', 'hero_class': 'Mage', 'maximal_lifepoints': 50, 'current_lifepoints': 50}
```

While we're at it, let us talk about default values in methods and functions

- We can also pass an **optional input**, the **maximal_lifepoints** value.
- If no value is passed for **maximal_lifepoints**, we initialize it, **by default**, to **100**.

```
1 class Hero:
2
3     def __init__(self, name, hero_class, maximal_lifepoints = 100):
4         """
5         Constructor function for the Hero class.
6         """
7         # Hero's name
8         self.name = name
9         # Hero's class
10        self.hero_class = hero_class
11
12        # Hero's maximal lifepoints
13        # (initialize as maximal_lifepoints)
14        self.maximal_lifepoints = maximal_lifepoints
15
16        # Hero's current lifepoints
17        # (initialize as maximal_lifepoints)
18        self.current_lifepoints = maximal_lifepoints
```

```
1 my_first_hero = Hero(name = "Sir Meowsalot", hero_class = "Warrior")
2 print(my_first_hero.__dict__)
```

```
{'name': 'Sir Meowsalot', 'hero_class': 'Warrior', 'maximal_lifepoints': 100, 'current_lifepoints': 100}
```

```
1 my_second_hero = Hero(name = "Lord Mustache", hero_class = "Mage", maximal_lifepoints = 50)
2 print(my_second_hero.__dict__)
```

```
{'name': 'Lord Mustache', 'hero_class': 'Mage', 'maximal_lifepoints': 50, 'current_lifepoints': 50}
```


The `__dict__` special attribute

Definition (`__dict__`):

Another useful concept is the **special dictionary attribute**, `__dict__`.

By default, it produces a **dictionary**

- containing all the **attributes** of your object and their currently assigned **values**.
- (Looks familiar? → OOT!)

```
1 class Hero:
2
3     def __init__(self):
4         '''
5         Constructor function for the Hero class.
6         '''
7         # Hero's name
8         self.name = "Sir Meowsalot"
9         # Hero's class
10        self.hero_class = "Warrior"
11        # Hero's maximal lifepoints
12        self.maximal_lifepoints = 100
13        # Hero's current lifepoints
14        self.current_lifepoints = 100
```

```
1 print(my_cat_hero.__dict__)
```

```
{'name': 'Sir Meowsalot', 'hero_class': 'Warrior', 'maximal_lifepoints': 100, 'current_lifepoints': 100}
```

Practice 1

Let us put these concept in practice with a first Activity.

- You will have to design several **custom classes of objects**.
 - Some of them will require **attributes** and **methods**.
 - Some of them will require to write **__init__** methods.
- Remember to control the values of attributes in your objects using the **__dict__** operation!

Class special methods

Definition (**special methods**):

A custom class may also have **special methods**.

- These methods have fixed names and are written with **double underscores (__)** before and after their names.
- These methods do something special when some basic operations (+, *, len(), etc.) are applied to our object.



About special methods

Definition (`__add__`):

There are more special methods, besides the `__init__` one. The `__add__` special method defines the behavior you want for your objects when two objects of our custom class are summed together with `+`.

- This allows us to control what the `+` behavior should be. This decided that `+` would be concatenation for strings/lists, math addition for int/float, etc.

```
1 class Coordinate:
2
3     def __init__(self, x = 0, y = 0):
4         self.x = x
5         self.y = y
6
7     def __add__(self, other):
8         new = Coordinate()
9         new.x = self.x + other.x
10        new.y = self.y + other.y
11        return new
```

```
1 A = Coordinate(3, 4)
2 B = Coordinate(1, 2)
3 C = A + B
4 print(type(C))
5 print(C.__dict__)
```

```
<class '__main__.Coordinate'>
{'x': 4, 'y': 6}
```

About special methods

Typically, we will have **special methods** for each **operator** (+, -, *, /, etc.), and **built-in function** (len, print, etc.).

- While it would be impossible for me to describe and showcase all of them one by one, you can easily find them in the documentation, here:

<https://docs.python.org/3/reference/datamodel.html#special-method-names>

Method	Result
<code>__add__(self, other)</code>	<code>self + other</code>
<code>__sub__(self, other)</code>	<code>self - other</code>
<code>__mul__(self, other)</code>	<code>self * other</code>
<code>__div__(self, other)</code>	<code>self / other</code>
<code>__truediv__(self, other)</code>	<code>self / other</code> (future)
<code>__floordiv__(self, other)</code>	<code>self // other</code>
<code>__mod__(self, other)</code>	<code>self % other</code>
<code>__divmod__(self, other)</code>	<code>divmod(self, other)</code>
<code>__pow__(self, other [, modulo])</code>	<code>self ** other</code> , <code>pow(self, other, modulo)</code>
<code>__lshift__(self, other)</code>	<code>self << other</code>
<code>__rshift__(self, other)</code>	<code>self >> other</code>
<code>__iadd__(self, other)</code>	<code>self += other</code>

The str special method

Definition (`__str__`):

The `__str__` special method defines what happens when you attempt to **convert** your custom object into a **string** type object.

- It is typically useful to decide what should be **displayed** on screen when you attempt to **print()** your object!

```
1 class Coordinate:
2
3     def __init__(self, x = 0, y = 0):
4         self.x = x
5         self.y = y
6
7     def __str__(self):
8         return "This is a Coordinate object with values x = {} and y = {}".format(self.x, self.y)
```

```
1 A = Coordinate(3, 4)
2 B = str(A)
3 print(type(B))
4 print(B)
```

```
<class 'str'>
This is a Coordinate object with values x = 3 and y = 4
```

```
1 print(A)
```

```
This is a Coordinate object with values x = 3 and y = 4
```

Call method

Definition (`__call__`):

The `__call__` special method defines what happens when you attempt to **call** your custom object and attempt to use it as a **function** and **pass it some arguments**.

- This also serves to show that functions are, technically, “just a special type of variable”.
- Functions defined with `def` could be seen as variables with a `__call__` method containing your instructions. (Technically a gross oversimplification, but ok for now).

```
1 class Coordinate:
2
3     def __init__(self, x = 0, y = 0):
4         self.x = x
5         self.y = y
6
7     def __call__(self, a, b, c):
8         val = a*self.x + b*self.y + c
9         return val
```

```
1 x = 3
2 y = 4
3 A = Coordinate(x, y)
4 a = 1
5 b = 2
6 c = 3
7 print(A(a, b, c))
8 print(a*x + b*y + c)
```

14

14

Practice 2

Let us put these concept in practice with a new Activity.

- You will reuse several **custom classes of objects from Activity 1**.
 - Some of them will require **some additional special methods**.
- Remember to control the values of attributes in your objects using the **__dict__** operation!

Our toy example: an RPG main protagonist

Let us get back to our **Hero** object.

- **Problem:** our Hero's attack capabilities probably depend on the weapon he has **equipped**.
- And this weapon should probably be an **object** as well!
- Maybe we should have variables of type *Weapon*, and have them interact with the *Hero* class?



Introducing a Weapon class object!

Reusing the previous concepts, we could define a **Weapon object**

It will have its own attributes, such as

- a name
- some attack values,
- and possibly more stuff.

(For now, let us keep it simple.)

```
1 class Weapon:
2
3     def __init__(self, name, attack):
4         self.name = name
5         self.attack = attack
```

```
1 my_sword = Weapon(name = "Sword of Blazing Justice", attack = 10)
2 print(my_sword.__dict__)
```

```
{'name': 'Sword of Blazing Justice', 'attack': 10}
```

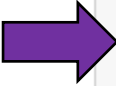
Equip a weapon!

(a.k.a the « has-a » relationship)

We can then have our Hero equip a Weapon object, with our own **equip_weapon()** method.

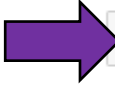
- It assigns a custom **Weapon object** (which we created earlier) to the **attribute equipped_weapon** of our **Hero object**.
- Our Hero can then equip a Weapon object, with our **equip_weapon()** method!

```
1 class Hero:
2
3     def __init__(self):
4         ...
5         Constructor function for the Hero class.
6         ...
7         # Hero's name
8         self.name = "Sir Meowsalot"
9         # Hero's equipped weapon
10        self.equipped_weapon = None
11
12    def equip_weapon(self, weapon_object):
13        ...
14        Equip weapon method
15        ...
16        # Assign weapon object to equipped_weapon attribute of Hero
17        self.equipped_weapon = weapon_object
```



```
1 my_cat_hero = Hero()
2 my_sword = Weapon(name = "Sword of Blazing Justice", attack = 10)
3 my_cat_hero.equip_weapon(my_sword)
4 print(my_cat_hero.__dict__)
```

{'name': 'Sir Meowsalot', 'equipped_weapon': <__main__.Weapon object at 0x000002DF7D65D0B8>}



```
1 print(my_cat_hero.equipped_weapon.__dict__)
```

{'name': 'Sword of Blazing Justice', 'attack': 10}

The « has-a » relationship

Definition (the “has-a” relationship in OOP):

In Object-Oriented Programming, this defines a “has-a” relationship between our Hero class and our Weapon class.

- We then say that our **Hero** object “has-a” **Weapon** object.
- Because one of our **Hero object’s attributes** is a **Weapon object**.



```

1 class Hero:
2
3     def __init__(self):
4         '''
5         Constructor function for the Hero class.
6         '''
7         # Hero's name
8         self.name = "Sir Meowsalot"
9         # Hero's class
10        self.hero_class = "Warrior"
11        # Hero's maximal lifepoints
12        self.maximal_lifepoints = 100
13        # Hero's current lifepoints
14        self.current_lifepoints = 100
15        # Hero's equipped weapon
16        self.equiped_weapon = None
17
18    def equip_weapon(self, weapon_object):
19        '''
20        Equip weapon method
21        '''
22        # Assign weapon object to equipped_weapon attribute of Hero
23        self.equiped_weapon = weapon_object

```

```

1 my_cat_hero = Hero()
2 my_sword = Weapon(name = "Sword of Blazing Justice", attack = 10)
3 my_cat_hero.equip_weapon(my_sword)
4 print(my_cat_hero.__dict__)

```

```
{'name': 'Sir Meowsalot', 'hero_class': 'Warrior', 'maximal_lifepoints': 100, 'current_lifepoints': 100, 'equiped_weapon': <__main__.Weapon object at 0x0000017F44B485F8>}
```

```
1 print(my_cat_hero.equiped_weapon.__dict__)
```

```
{'name': 'Sword of Blazing Justice', 'attack': 10}
```

Practice 3

Let us put these concept in practice with a new Activity.

- You will reuse several **custom classes of objects from Activity 1&2**.
- You will have to implement new additional classes and establish some **“has-a” relationships** between the different classes.
- Remember to control the values of attributes in your objects using the **__dict__** operation!

Modifying our Weapon class object

Earlier, we defined a Weapon object, with attributes such as name and attack values.

- **Problem:** how do we efficiently take into account multiple types of weapons possibilities? (sword, bow, axe, magic staff, etc.

```
1 class Weapon:
2
3     def __init__(self, name, attack):
4         self.name = name
5         self.attack = attack
```

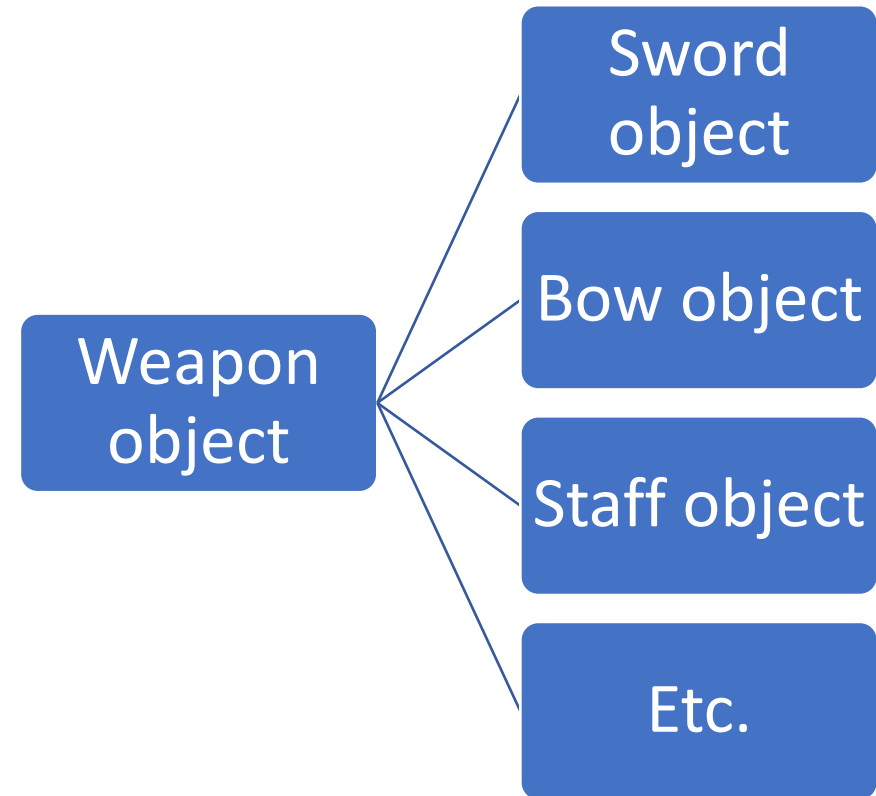
```
1 my_sword = Weapon(name = "Sword of Blazing Justice", attack = 10)
2 print(my_sword.__dict__)
```

```
{'name': 'Sword of Blazing Justice', 'attack': 10}
```

Objects and sub-classes of objects

Typically, our Hero could equip a Weapon object...

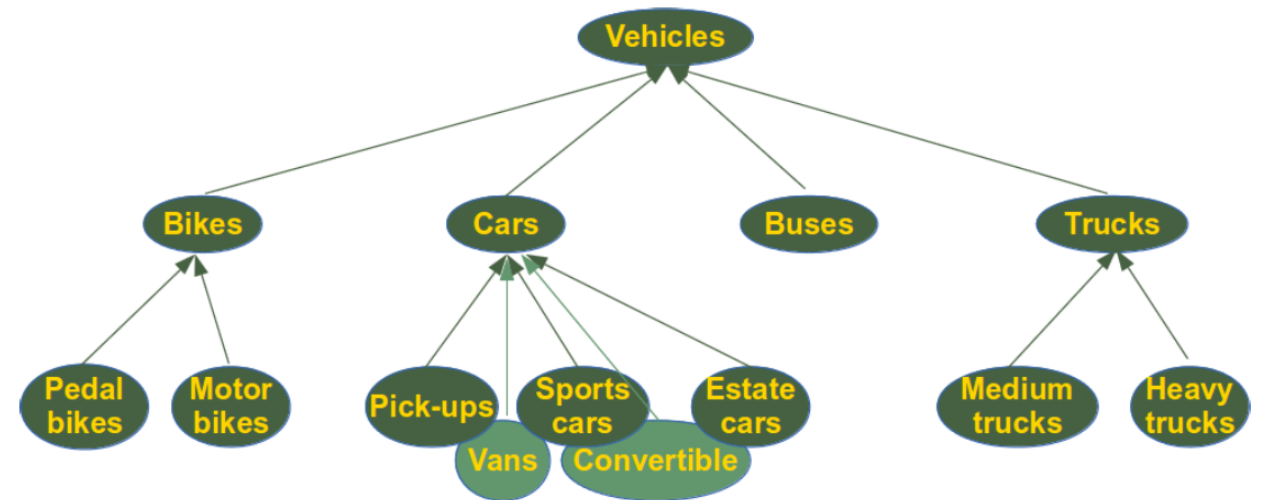
- But it could be a **sword**, a **bow**, a **staff**, etc.
- These weapons will probably have attributes and methods in common.
- But they might also have different attributes and methods, as our Hero will probably operate those weapons differently.



Objects and sub-classes of objects

In fact, it is common in life to have **objects** and **sub-classes** of objects.

- Cars, buses and trucks probably have some **attributes and methods** in common, because they are **vehicles objects**.
- But they probably have **attributes and methods** that are **specific** to them.



Introducing inheritance!

(a.k.a. the « is-a » relationship)

Definition (**Inheritance**):

It is possible to **create** a class, which **reuses methods** and **attributes** from another class!

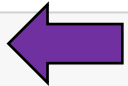
- In OOP, this is called **inheritance**.
- We simply mention the name of the previous class in the class **definition**!



```
1 class Sword(Weapon):
2
3     def __init__(self, name, attack):
4         # Reuse the Weapon object __init__ method!
5         Weapon.__init__(self, name, attack)
6         # Add extra attributes, specific to this weapon
7         self.weapon_type = 'Sword'
8         self.weapon_range = '3'
9
10    def slash(self):
11        print("A big slash to the face!")
```

```
1 my_sword = Sword(name = "Sword of Blazing Justice", attack = 10)
2 print(my_sword.__dict__)
```

```
{'name': 'Sword of Blazing Justice', 'attack': 10, 'weapon_type': 'Sword', 'weapon_range': '3'}
```



```
1 class Staff(Weapon):
2
3     def __init__(self, name, attack):
4         # Reuse the Weapon object __init__ method!
5         Weapon.__init__(self, name, attack)
6         # Add extra attributes, specific to this weapon
7         self.weapon_type = 'Staff'
8         self.weapon_range = '25'
9
10    def cast_fireball(self):
11        print("PEW PEW PEW!")
```

```
1 my_staff = Staff(name = "Staff of Impeccable Fireworks", attack = 5)
```

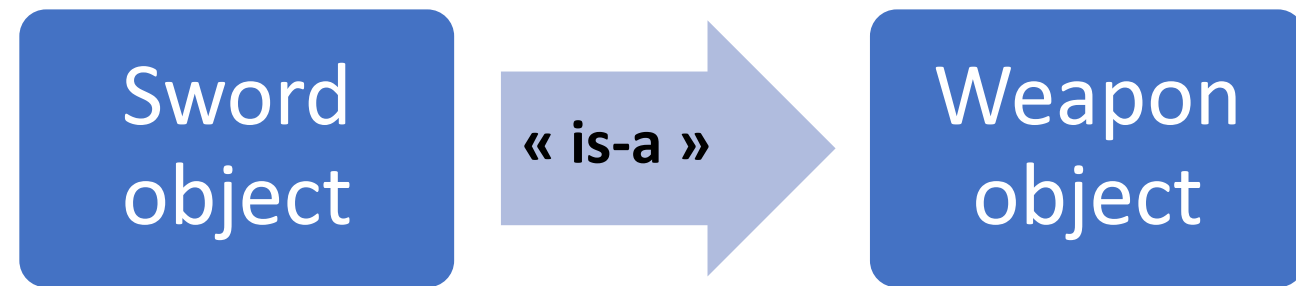
Inheritance and « is-a » relationship

Inheritance is very useful

- it allows for **code reuse**
- and **better architecture** of the objects in your code!

Inheritance defines a « is-a » relationship between objects.

We say that Sword is the **child class** and Weapon is the **mother class** in that case. We might also refer to them as **derived class** and **base class**.



Inheritance in practice

With **inheritance**, it is possible to **create** a class, which **reuses methods** and **attributes** from another class.

- We simply mention the name of the previous class in the class **definition**.
- And reuse the **__init__ constructor** from the previous class in the **__init__ constructor** of new class.

```
1 class Sword(Weapon):
2
3     def __init__(self, name, attack):
4         # Reuse the Weapon object __init__ method!
5         Weapon.__init__(self, name, attack)
6         # Add extra attributes, specific to this weapon
7         self.weapon_type = 'Sword'
8         self.weapon_range = '3'
9
10    def slash(self):
11        print("A big slash to the face!")
```

```
1 my_sword = Sword(name = "Sword of Blazing Justice", attack = 10)
2 print(my_sword.__dict__)
```

```
{'name': 'Sword of Blazing Justice', 'attack': 10, 'weapon_type': 'Sword', 'weapon_range': '3'}
```

```
1 class Staff(Weapon):
2
3     def __init__(self, name, attack):
4         # Reuse the Weapon object __init__ method!
5         Weapon.__init__(self, name, attack)
6         # Add extra attributes, specific to this weapon
7         self.weapon_type = 'Staff'
8         self.weapon_range = '25'
9
10    def cast_fireball(self):
11        print("PEW PEW PEW!")
```

```
1 my_staff = Staff(name = "Staff of Impeccable Fireworks", attack = 5)
```

Inheritance in practice

On top of attributes, inheritance allows for methods reuse.

- In addition, methods from the parent class can be reused freely by the child class.
- If both the child class and the parent class have a method with the same name, the child class will override the method definition for objects of the child class.

```
# Parent class
class Weapon:
    def __init__(self, name, damage):
        self.name = name
        self.damage = damage
    def describe(self):
        return f"This is a weapon, with name {self.name} and it has {self.damage} damage points"
    def attack(self):
        return f"The {self.name} deals {self.damage} damage. BAM."

# Child class
class Sword(Weapon):
    def __init__(self, name, damage, blade_length):
        super().__init__(name, damage) # Reusing the parent class's __init__ method
    # Overriding the attack method in the parent class
    def attack(self):
        return f"The {self.name} slashes for {self.damage} damage. SLASH!"

# Creating an instance of the Weapon class
weapon = Weapon("Generic Weapon", 10)
print(weapon.attack())
# Creating an instance of the Sword class
sword = Sword("Excalibur", 50, 120)
print(sword.describe()) # Will use the parent class method.
print(sword.attack()) # Will override and use the child class method instead.

The Generic Weapon deals 10 damage. BAM.
This is a weapon, with name Excalibur and it has 50 damage points
The Excalibur slashes for 50 damage. SLASH!
```

(Will enlarge in the next slide.)

In

Or
all

- It

p
k

- It

p
t

C

C

```
# Parent class
class Weapon:
    def __init__(self, name, damage):
        self.name = name
        self.damage = damage
    def describe(self):
        return f"This is a weapon, with name {self.name} and it has {self.damage} damage points"
    def attack(self):
        return f"The {self.name} deals {self.damage} damage. BAM."

# Child class
class Sword(Weapon):
    def __init__(self, name, damage, blade_length):
        super().__init__(name, damage) # Reusing the parent class's __init__ method
        # Overriding the attack method in the parent class
    def attack(self):
        return f"The {self.name} slashes for {self.damage} damage. SLASH!"

# Creating an instance of the Weapon class
weapon = Weapon("Generic Weapon", 10)
print(weapon.attack())
# Creating an instance of the Sword class
sword = Sword("Excalibur", 50, 120)
print(sword.describe()) # Will use the parent class method.
print(sword.attack()) # Will override and use the child class method instead.
```

The Generic Weapon deals 10 damage. BAM.

This is a weapon, with name Excalibur and it has 50 damage points

The Excalibur slashes for 50 damage. SLASH!

Practice 4

Let us put these concept in practice with a new Activity.

- You will reuse several **custom classes of objects from Activity 1&2.**
- You will have to implement new additional classes and establish some **“is-a” relationships** between the different classes.
- Remember to control the values of attributes in your objects using the **__dict__** operation!

Some good practice in OOP

As mentionned earlier, using `__init__` **constructors** for your class is considered **good** practice.

- In addition, it could also be interesting to use **setters** and **getters methods** for your class **attributes**.
- And also, to define if your class **attributes** should be **public** or **private**.

Display lifepoints method

Let us say we want to design a method that prints the current lifepoints of our Hero on screen.

A possible way to do it is this shown on the right.

- And it works just fine!
- However, this is considered **bad practice**.

```
1 class Hero:
2
3     def __init__(self):
4         '''
5         Constructor function for the Hero class.
6         '''
7         # Hero's name
8         self.name = "Sir Meowsalot"
9         # Hero's class
10        self.hero_class = "Warrior"
11        # Hero's maximal lifepoints
12        self.maximal_lifepoints = 100
13        # Hero's current lifepoints
14        self.current_lifepoints = 100
15
16
17    def display_current_life(self):
18        '''
19        A method calling some attributes of the class.
20        '''
21        # Print to let the player know about its current life total
22        print("Your hero has {} lifepoints.".format(self.current_lifepoints))
```

```
1 my_cat_hero = Hero()
```

```
1 my_cat_hero.display_current_life()
```

Your hero has 100 lifepoints.

Setters and getters

- **Good practice:** design methods for getting and setting attributes of a class. **A method should get or set attributes, but not both!**
- **Getter** method: fetches the current value stored in attribute.
- Here, we demonstrate an example of a **getter** method.
 - And apply it in our `display_current_life` method.

```
1 class Hero:
2
3     def __init__(self):
4         """
5         Constructor function for the Hero class.
6         """
7         # Hero's name
8         self.name = "Sir Meowsalot"
9         # Hero's class
10        self.hero_class = "Warrior"
11        # Hero's maximal lifepoints
12        self.maximal_lifepoints = 100
13        # Hero's current lifepoints
14        self.current_lifepoints = 100
15
16    def get_current_lifepoints(self):
17        """
18        A getter method returning the value of the current lifepoints.
19        """
20        print("Getter called for current_lifepoints")
21        return self.current_lifepoints
22
23    def display_current_life(self):
24        """
25        A method calling some attributes of the class.
26        """
27        # Print to let the player know about its current life total
28        print("Your hero has {} lifepoints.".format(self.get_current_lifepoints()))

```

```
1 my_cat_hero = Hero()

```

```
1 my_cat_hero.display_current_life()

```

Getter called for current_lifepoints
Your hero has 100 lifepoints.

Why is it good practice to use setters and getters?

But, why would I write a getter method?!

- It seems cumbersome for no reason!

```
1 class Hero:
2
3     def __init__(self):
4         """
5         Constructor function for the Hero class.
6         """
7         # Hero's name
8         self.name = "Sir Meowsalot"
9         # Hero's class
10        self.hero_class = "Warrior"
11        # Hero's maximal lifepoints
12        self.maximal_lifepoints = 100
13        # Hero's current lifepoints
14        self.current_lifepoints = 100
15
16    def get_current_lifepoints(self):
17        """
18        A getter method returning the value of the current lifepoints.
19        """
20        print("Getter called for current_lifepoints")
21        return self.current_lifepoints
22
23    def display_current_life(self):
24        """
25        A method calling some attributes of the class.
26        """
27        # Print to let the player know about its current life total
28        print("Your hero has {} lifepoints.".format(self.get_current_lifepoints()))
```

```
1 my_cat_hero = Hero()
```

```
1 my_cat_hero.display_current_life()
```

```
Getter called for current_lifepoints
Your hero has 100 lifepoints.
```

Why is it good practice to use setters and getters?

Question: But, why would I write a getter method?!

- It seems cumbersome for no reason!

Actually, it makes your code **more modular and stable!**

```
1 class Hero:
2
3     def __init__(self):
4         """
5         Constructor function for the Hero class.
6         """
7         # Hero's name
8         self.name = "Sir Meowsalot"
9         # Hero's class
10        self.hero_class = "Warrior"
11        # Hero's maximal lifepoints
12        self.maximal_lifepoints = 100
13        # Hero's current lifepoints
14        self.current_lifepoints = 100
15
16    def get_current_lifepoints(self):
17        """
18        A getter method returning the value of the current lifepoints.
19        """
20        print("Getter called for current_lifepoints")
21        return self.current_lifepoints
22
23    def display_current_life(self):
24        """
25        A method calling some attributes of the class.
26        """
27        # Print to let the player know about its current life total
28        print("Your hero has {} lifepoints.".format(self.get_current_lifepoints()))
```

```
1 my_cat_hero = Hero()
```

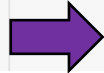
```
1 my_cat_hero.display_current_life()
```

```
Getter called for current_lifepoints
Your hero has 100 lifepoints.
```

Why is it good practice to use setters and getters?

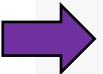
- Let us pretend that – for some reason – the dev team in charge of the lifepoints, decided to change the way it is stored in memory.
- No longer stored as lifepoints numbers, but as a lifepoint percentage.
- Our methods no longer work!

```
1 class Hero:
2
3     def __init__(self):
4         """
5         Constructor function for the Hero class.
6         """
7         # Hero's name
8         self.name = "Sir Meowsalot"
9         # Hero's class
10        self.hero_class = "Warrior"
11        # Hero's maximal lifepoints
12        self.maximal_lifepoints = 100
13        # Hero's current lifepoints
14        self.current_life_percentage = 100
15
```



```
def get_current_lifepoints(self):
    """
    A getter method returning the value of the current lifepoints.
    """
    print("Getter called for current_lifepoints")
    return self.current_lifepoints

def display_current_life(self):
    """
    A third method calling some attributes of the class.
    """
    # Print to let the player know about its current life total
    print("Your hero has {} lifepoints.".format(self.get_current_lifepoints()))
```



Why is it good practice to use setters and getters?

- Let us pretend that – for some reason – the dev team in charge of the lifepoints, decided to change the way it is stored in memory.
- No longer stored as lifepoints numbers, but as a lifepoint percentage.
- Our methods no longer work!
- But, only **one line** to change to make it work again!

```
1 class Hero:
2
3     def __init__(self):
4         """
5         Constructor function for the Hero class.
6         """
7         # Hero's name
8         self.name = "Sir Meowsalot"
9         # Hero's class
10        self.hero_class = "Warrior"
11        # Hero's maximal lifepoints
12        self.maximal_lifepoints = 100
13        # Hero's current lifepoints
14        self.current_life_percentage = 100
15
16    def get_current_lifepoints(self):
17        """
18        A getter method returning the value of the current_lifepoints,
19        based on maximal_lifepoints and current_life_percentage.
20        """
21        print("Getter called for current_lifepoints")
22        return round(self.current_life_percentage*self.maximal_lifepoints/100)
23
24    def display_current_life(self):
25        """
26        A third method calling some attributes of the class.
27        """
28        # Print to let the player know about its current life total
29        print("Your hero has {} lifepoints.".format(self.get_current_lifepoints()))
```



```
1 my_cat_hero = Hero()
```

```
1 my_cat_hero.display_current_life()
```

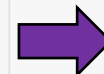
Getter called for current_lifepoints
Your hero has 100 lifepoints.

Why is it good practice to use setters and getters?

Lesson: using setters and getters for attributes will make your code more modular and robust to change.

- It is therefore considered **good practice**.

```
1 class Hero:
2
3     def __init__(self):
4         """
5         Constructor function for the Hero class.
6         """
7         # Hero's name
8         self.name = "Sir Meowsalot"
9         # Hero's class
10        self.hero_class = "Warrior"
11        # Hero's maximal lifepoints
12        self.maximal_lifepoints = 100
13        # Hero's current lifepoints
14        self.current_life_percentage = 100
15
16    def get_current_lifepoints(self):
17        """
18        A getter method returning the value of the current_lifepoints,
19        based on maximal_lifepoints and current_life_percentage.
20        """
21        print("Getter called for current_lifepoints")
22        return round(self.current_life_percentage*self.maximal_lifepoints/100)
23
24    def display_current_life(self):
25        """
26        A third method calling some attributes of the class.
27        """
28        # Print to let the player know about its current life total
29        print("Your hero has {} lifepoints.".format(self.get_current_lifepoints()))
```



```
1 my_cat_hero = Hero()
```

```
1 my_cat_hero.display_current_life()
```

Getter called for current_lifepoints
Your hero has 100 lifepoints.

Defining public, protected and private attributes

Another good practice consists of defining **public**, **protected** and **private** attributes.

- **Public:** everyone can modify the attribute.
- **Protected:** public, but refrain from modifying it from outside the class.
- **Private:** only the functions called within my object can modify the attribute.

Type of attribute	Public	Protected	Private
Can be modified within the class methods.	Yes	Yes	Yes
Can be modified by function and methods outside of the class.	Yes	Yes	No
It is acceptable for this attribute to be modified by functions and methods outside the class.	Yes	No	Does not apply

Private attributes

Private: only the functions called within my object can modify the attribute.

- We can **make** an attribute **private** by adding a double **underscore** (**__**) in front of its name.
- Calling it or modifying it has no effect.

```
1 class Hero:
2
3     def __init__(self):
4         ...
5         Constructor function for the Hero class.
6         ...
7         # Hero's name
8         self.__name = "Sir Meowsalot"
9         # Hero's class
10        self.__hero_class = "Warrior"
11        # Hero's maximal lifepoints
12        self.maximal_lifepoints = 100
13        # Hero's current lifepoints
14        self.current_life_percentage = 100
15
16    def get_hero_name(self):
17        return self.__name
```

```
1 my_cat_hero = Hero()
```

```
1 # This works, because it's a public attribute
2 my_cat_hero.current_life_percentage = 50
3 print(my_cat_hero.current_life_percentage)
```

50

```
1 # This works
2 print(my_cat_hero.get_hero_name())
3 # This does not work
4 print(my_cat_hero.__name)
```

Sir Meowsalot

```
-----
AttributeError                                Traceback (most recent call last)
<ipython-input-129-7f4b66a9de98> in <module>
      2 print(my_cat_hero.get_hero_name())
      3 # This does not work
----> 4 print(my_cat_hero.__name)

AttributeError: 'Hero' object has no attribute '__name'
```

```
1 # Trying to change the name from outside the class leaves it unaffected
2 my_cat_hero.__name = "Poop-eater"
3 print(my_cat_hero.get_hero_name())
```

Sir Meowsalot

Protected attributes

Protected: public, but should refrain from modifying it from outside the class.

- We can **make** an attribute **protected** by adding a single **underscore** (**_**) in front of its name.
- More of an indication to other devs working in your team!

```
1 class Hero:
2
3     def __init__(self):
4         ...
5         Constructor function for the Hero class.
6         ...
7         # Hero's name
8         self._name = "Sir Meowsalot"
9         # Hero's class
10        self.hero_class = "Warrior"
11        # Hero's maximal lifepoints
12        self.maximal_lifepoints = 100
13        # Hero's current lifepoints
14        self.current_life_percentage = 100
15
16    def get_hero_name(self):
17        return self._name
18
19    def set_hero_name(self, value):
20        print("Warning: hey, the name attribute is protected, don't do that!")
21        self._name = value
```

```
1 my_cat_hero = Hero()
```

```
1 # This works
2 print(my_cat_hero.get_hero_name())
```

→ Sir Meowsalot

```
1 # Trying to change the name from outside the class leaves it unaffected
2 my_cat_hero.set_hero_name("Poop-eater")
3 print(my_cat_hero.get_hero_name())
```

→ Warning: hey, the name attribute is protected, don't do that!
Poop-eater

Merging everything, by setting attributes properties using the **property** keyword

Last but not least, it is also
considered good practice to
create properties.

- i.e. default setter/getter
methods for each attribute.
- It is done with the **property**
keyword.

```
1 class Hero:
2
3     def __init__(self):
4         # Constructor function for the Hero class.
5         # Hero's name
6         self.__name = "Sir Meowsalot"
7         # Hero's class
8         self.__hero_class = "Warrior"
9         # Hero's maximal lifepoints
10        self._maximal_lifepoints = 100
11        # Hero's current lifepoints
12        self._current_lifepoints = 100
13
14    def set_current_lifepoints(self, value):
15        # A setter method setting the current lifepoints to value.
16        print("Setter called for attribute _current_lifepoints")
17        self._current_lifepoints = value
18
19    def get_current_lifepoints(self):
20        # A getter method returning the value of the current lifepoints.
21        print("Getter called for attribute _current_lifepoints")
22        return self._current_lifepoints
23
24    ...
25    Set property for the current_lifepoints attribute
26    ...
27    current_lifepoints = property(get_current_lifepoints, set_current_lifepoints)
```



Merging everything, by setting attributes properties using the **property** keyword

- Thanks to the **property** keyword...
- Whenever we execute ***my_cat_hero.current_lifepoints***, we now automatically call the **getter** method, i.e.

my_cat_hero.get_current_lifepoints()

```
1 class Hero:
2
3     def __init__(self):
4         # Constructor function for the Hero class.
5         # Hero's name
6         self.__name = "Sir Meowsalot"
7         # Hero's class
8         self.__hero_class = "Warrior"
9         # Hero's maximal lifepoints
10        self._maximal_lifepoints = 100
11        # Hero's current lifepoints
12        self._current_lifepoints = 100
13
14    def set_current_lifepoints(self, value):
15        # A setter method setting the current lifepoints to value.
16        print("Setter called for attribute _current_lifepoints")
17        self._current_lifepoints = value
18
19    def get_current_lifepoints(self):
20        # A getter method returning the value of the current lifepoints.
21        print("Getter called for attribute _current_lifepoints")
22        return self._current_lifepoints
23
24    '''
25    Set property for the current_lifepoints attribute
26    '''
27    current_lifepoints = property(get_current_lifepoints, set_current_lifepoints)
```

```
1 my_cat_hero = Hero()
```

```
1 print(my_cat_hero.current_lifepoints)
```

Getter called for attribute _current_lifepoints
100

Merging everything, by setting attributes properties using the **property** keyword

- And similarly, with the **setter** method, when we try to assign a value to our attribute.

```
1 class Hero:
2
3     def __init__(self):
4         # Constructor function for the Hero class.
5         # Hero's name
6         self.__name = "Sir Meowsalot"
7         # Hero's class
8         self.__hero_class = "Warrior"
9         # Hero's maximal lifepoints
10        self._maximal_lifepoints = 100
11        # Hero's current lifepoints
12        self._current_lifepoints = 100
13
14        def set_current_lifepoints(self, value):
15            # A setter method setting the current lifepoints to value.
16            print("Setter called for attribute _current_lifepoints")
17            self._current_lifepoints = value
18
19        def get_current_lifepoints(self):
20            # A getter method returning the value of the current lifepoints.
21            print("Getter called for attribute _current_lifepoints")
22            return self._current_lifepoints
23
24        ...
25        Set property for the current_lifepoints attribute
26        ...
27        current_lifepoints = property(get_current_lifepoints, set_current_lifepoints)
```

```
1 my_cat_hero = Hero()
```

```
1 print(my_cat_hero.current_lifepoints)
```

```
Getter called for attribute _current_lifepoints
100
```

```
1 my_cat_hero.current_lifepoints = 50
2 print(my_cat_hero.current_lifepoints)
```

```
Setter called for attribute _current_lifepoints
Getter called for attribute _current_lifepoints
50
```

Feel free to explore and play with OOP concepts!

They will bring your coding capabilities to great heights!

(At this point, you should also start to recognize how your favorite games/apps have probably been coded, and maybe even attempt to code them yourself!)



Conclusion (Chapter 17)

- What are **objects** and **classes**? How do they relate to **dictionaries**?
- What are **attributes** and **methods** in a class?
- What is the **__init__** constructor method?
- What is the **__dict__** special attribute?
- What are **special methods** in Python?
- What is the “**has-a**” relationship in OOP?
- (*What is the “**is-a**” relationship in OOP?*)
- (*What are the different **attributes privacies**?*)
- (*What are **setters, getters and properties**?*)